

23/6/25

Python Programming Assignment - 6

1) What is a user-defined function in Python? Why do we need functions instead of writing code repeatedly?

Ans: The function which is written by the user or programmer by itself and is not predefined or inbuilt function. This type of function is known as user-defined function.

User has to be a ~~pro~~ better programmer so that it should be the best asset to the organization.

For this the written programs/code should be neat, clean and executable also ~~to show~~ should use less memory and work efficiently at the runtime. Let's elaborate this with an example.

Suppose, the problem statement says that user has to accept values multiple times to generate different values each time and the thing program will do is the ^{calculations} Addition.

So, this concept known as user-defined function is used in this. The programmer will first define all the functions (like addition, Sub, Div, Multi, etc) and it so that when user will enter an integer value and ask the program to do any specific operation like add two numbers and output will be generated. Therefore, in this case the program will call the required function which is user-defined and calculates it to show the output.

In good programming practice, organization expects that programmer should write less number of line code and generate the output so if any program needs to call

or do a specific task 2-3 times or so on, writing code repeatedly is not the solution. Here, programmer should define ~~the~~ one function which will include that task and called that function ^{below} when ~~or~~ when needed this save effort^{and time} of writing code repeatedly.

2) Why should excessive use of global variables be avoided in large programs?

Ans:

2) Explain the syntax of a function definition using the def keyword. Why is the colon (:) mandatory?

Ans: Syntax:

```
def Demo():  
    print("Jay Ganesh")
```

def keyword tells Python that I am defining a function now which is also a user-defined function. This function is created first, and it is executed only when called.

The colon (:) is mandatory because, in Python (colon) means it starts a block of code. Statements that opens a new defined scope must ends its header with a colon to explicitly tell the parser: Next line must be indented properly to our code.

3) What is the importance of indentation inside a function body? What happens if indentation is incorrect?

Ans: Python uses indentation to define blocks (unlike C/C++ which uses `{ }`). Python uses tabs / white spaces. Indentation indicates the exact entry and exit points of the function's ~~excec~~ execution block.

If indentation is incorrect, the Python parser catches it at compile-time (before the code even runs) and raises a Syntax Error: Indentation Error.

4) Difference between calling a function exist without any statements inside its body? If yes, how?

Ans:

4) Differentiate between calling a function and defining a function with a simple explanation.

Ans: The difference between calling and defining function is:

- Defining a function is declaration statement. When Python reads `def my-func():`, it compiles the code inside the block into bytecode, allocates memory for the function object, and attaches it to the identifier `my-func` in the current namespace. No internal logic is executed.
- Calling a function (`my-func()`) is an expression. It instructs the interpreter to resolve the name `my-func`, push a new stack frame onto the call stack, pass control flow into that memory address, execute the bytecode, and pop the frame of the stack when finished.

5) Can a Python function exist without any statements inside its body? If yes, how?

Ans: Yes, Python function exists without any statements inside its body. Since Python relies on indentation to define code blocks, an empty block will throw a `IndentationError` or `SyntaxError`. To bypass this, we must use a placeholder. One of the placeholder is 'pass' statement. The pass keyword is a null operation. It acts as a valid placeholder when a statement is required syntactically, but you don't want any code to execute.

```
def fun():
    pass
```

6) What is the role of the pass statement in function definition?

Ans: The pass statement is a null operation. When the Python interpreter hits pass, it literally does nothing and moves to the next line. Its sole role is to satisfy the interpreter's syntactical requirement for an indented block. It acts as a valid placeholder when a statement is required syntactically, but we don't want any code to execute.

7) What is the difference between parameters and arguments in Python functions?

Ans: Parameters are the named variables defined in the function's declaration. The use of these variable is to receive data and store it. eg: `(def Mult(x,y))` where `x` & `y` are the parameter which will receive value in it.

Arguments are the actual values passed into the function at the moment of process. (eg. `points(10, 25)` where `10` & `25` are the argument values which will be passed to the parameters.

In Python, when an argument is passed to a parameter, the parameter becomes a new reference pointing to the exact same object in memory as the argument.

8) What happens if the number of arguments passed does not match the function parameters?

Ans: If the number of arguments passed does not match the function parameter, Python will raise a `TypeError` at runtime.

If the user is passing Too few Arguments, Python maps your arguments to the parameters from left to right. The moment it runs out of arguments before filling every parameter, it crashes.

eg:

```
def fun(name, age):  
    print("Created {name}, age {age}")
```

```
fun("sara_99")
```

Output:

```
TypeError: create_user()fun() missing 1 required positional argument: 'age'
```

If the user is passing Too many Arguments, then it has "parking spots" for it, it refuses to dump the extra data on the ground. It rejects the whole call.

eg:

```
def fun(name):
```

```
    print("Hello, ", name)
```

```
fun("Marvellous", "Python")
```

Output:

TypeError: fun() takes 1 positional argument but 2 were given.

9) What is return value in a function?

Ans:

return value is in a function send the value back to the caller. The moment the interpreter sees the return keyword, it evaluates the expression next to it, terminates the local scope, destroys the local variables, and hands the value back to the caller.

And if a function does not use return, it returns None type value by default. by the end of the block.

10) Can a Python function return multiple values? If yes, how does the Python handle it internally?

Ans:

Yes, a Python function can return multiple values. However, in Python function returns only a single object. But when you separate multiple values with comma in the return statement, it automatically bundles them into a single tuple data structure.

When we write return a, b, c Python uses a feature called Tuple Packing. Because the expressions are separated by commas, the interpreter automatically packs those three variables into a single, immutable tuple object, and returns that one tuple. When the developer catches it on the outside

eg:

```
def Marvellous (No1, No2):  
    print ("Inside Marvellous: ", No1, No2)  
    return 21, 51
```

```
def main():
```

```
    Ret1, Ret2 = Marvellous (10, 20)  
    print ("Return values are: ", Ret1, Ret2)
```

```
main()
```